Movie Recommendation System

CSIT 557 — Advanced Techniques in Data Science | Spring 2026
Author: Md Allama Ikbal Sijan

---

1. Introduction

A recommendation system is a type of information filtering system that predicts what a user might be interested in based on past behavior. These systems are used everywhere: Netflix recommends movies, Amazon recommends products, and Spotify recommends songs. The core challenge is that users interact with only a small fraction of available items, leaving most of the user-item matrix unobserved. The goal is to fill in those missing entries as accurately as possible.

This project builds a movie recommendation system using the CiaoDVD dataset. Four models are implemented and compared: a global average baseline, user-based collaborative filtering, item-based collaborative filtering, and SVD matrix factorization. The project follows a vibe coding approach, using AI-assisted development throughout.

2. Dataset and Exploratory Data Analysis

2.1 Dataset Description

CiaoDVD is a real-world dataset crawled from dvd.ciao.co.uk in December 2013. It contains user ratings for DVD movies on a 1 to 5 star scale, including half-star increments. The dataset also includes a trust network between users. For this project, only the ratings file was used for model training and evaluation; the trust file was loaded for inspection.

Metric	Value
Total users	1,508
Total movies	2,071
Total ratings	35,497
Matrix sparsity	98.86%
Median ratings per user	15
Median ratings per movie	2

2.2 Key Findings

The rating matrix is extremely sparse at 98.86%, meaning the vast majority of user-movie pairs have no rating. This is a core challenge for any recommendation algorithm. User activity follows a strong long-tail pattern: the median user has rated only 15 movies, but the most active user has rated 237. Movies are even more skewed, with a median of just 2 ratings per movie and only a small number of popular titles driving most of the data. Ratings include half-star values rather than just whole integers as the dataset documentation suggests, and they skew toward 4 and 5 stars, indicating a positivity bias common in voluntary review datasets.

3. Models

3.1 Global Average Baseline

The simplest possible approach: predict the global mean rating from the training set for every user-movie pair. There is no personalization whatsoever. This model serves as a lower-bound benchmark. Any model that cannot beat it is not adding value.

3.2 User-Based Collaborative Filtering

The intuition is that users who agreed in the past will agree in the future. For a given user-movie pair, the model finds the K most similar users using cosine similarity, then predicts the rating as a weighted average of those neighbors' ratings on that movie. When a user or movie is unseen in training, the model falls back to the global mean. The number of neighbors K was tuned over [10, 20, 40] using a validation sample.

3.3 Item-Based Collaborative Filtering

Instead of finding similar users, this model finds similar movies. For each movie, it computes cosine similarity against all other movies based on their rating patterns. To predict a rating, it takes a weighted average of the user's ratings on movies similar to the target movie. Item-based CF tends to be more stable than user-based CF in sparse datasets because item rating patterns change more slowly than user behavior. K was tuned the same way as user-based CF.

3.4 SVD Matrix Factorization

SVD decomposes the user-item rating matrix into three matrices: U (users), sigma (importance weights), and Vt (movies). The key insight is that this produces a compact, dense representation of both users and movies in a shared latent space. Instead of relying on direct rating overlap, SVD captures hidden patterns: a user who likes action movies will have a similar latent vector to other action fans, even if they haven't rated the same specific films. The matrix was mean-centered before decomposition to remove user rating bias. The number of latent factors was tuned over [5, 10, 20, 50, 100], with k=10 giving the best validation RMSE.

4. Results

4.1 RMSE and MAE

Model	RMSE	MAE
Global Average	0.8122	0.6603
User-Based CF	0.8260	0.6605
Item-Based CF	0.7292	0.5726
SVD (k=10)	0.7235	0.5654

4.2 Precision@10 and Recall@10

A rating of 4.0 or above was used as the relevance threshold. Only users with at least 10 test ratings were included in this evaluation.

Model	Precision@10	Recall@10
Global Average	0.2366	0.6362
Item-Based CF	0.2502	0.6957
SVD (k=10)	0.2488	0.6863

4.3 Discussion

SVD achieved the best rating prediction accuracy with an RMSE of 0.7235 and MAE of 0.5654. This is expected because SVD learns latent factors that generalize across the sparse matrix, rather than relying on direct rating overlap between users or items. Item-based CF came in a close second (RMSE 0.7292), which suggests that movie-to-movie similarity is more stable than user-to-user similarity in this dataset. User-based CF performed no better than the global average baseline, most likely because most users have rated very few movies (median = 15), making it hard to find reliable neighbors.

On ranking quality, Item-Based CF achieved the highest Precision@10 (0.2502) and Recall@10 (0.6957), narrowly beating SVD despite SVD having better RMSE and MAE. This highlights an important distinction: a model that minimizes prediction error does not always rank the right items at the top of the list. Precision is low across all models (~25%) because recommending only 10 items out of thousands is inherently difficult. Recall is high (~69%) because users in this dataset have few test ratings, so hitting a few relevant items covers a large fraction of their liked movies.

5. Vibe Coding Reflection

This section is written entirely in my own words.

How I used AI assistants

I used AI assistants mainly for code generation. For each task, I gave the model a specific goal, the algorithm I wanted to implement, the steps I planned to follow, and the tuning approach. I then asked it to write comprehensive code based on those inputs. I also used it heavily for debugging: instead of spending hours searching Stack Overflow or Reddit, I pasted the terminal error output directly and got a fix in seconds.

Which prompts worked well and which did not

Prompts with chain-of-thought structure worked well, where I broke the task down into steps before asking for code. Detailed prompts with specific information, like the Python version, the library to use, and the exact columns in my dataset, also produced much better results. Asking the AI to ask clarifying questions before writing code was surprisingly effective because it caught gaps in my prompt that I hadn't noticed. Prompts without enough context consistently produced code that didn't fit my setup and needed multiple rounds of correction.

What I learned from the vibe coding process

Vibe coding is useful as long as you can read and understand the code that comes back. It is not a replacement for knowing the framework. Debugging became much faster. Previously, tracking down a specific error meant searching through forum threads and hoping someone had the same setup. Now I can paste the full terminal output and get a targeted fix. That alone saved significant time on this project.

A moment where AI-generated code surprised me

The AI-generated code was accurate on the first try more often than I expected. But what stood out more was a negative surprise: the AI does not understand the nuances of your local environment. It doesn't know your Python version, your terminal config, or your VS Code setup. The Surprise library compatibility issue with Python 3.13 is a good example. The AI kept suggesting installs that would never work on my machine until I gave it the exact error message and the version information. The lesson is that structured input and output verification are your responsibility, especially before putting any code into production. For building and experimentation though, it is genuinely a game changer.

How my prompting strategy changed

The strategy itself didn't change much, but I got better at reading the situation and choosing the right type of prompt faster. I also used a single chat session with the project rubric loaded as context from the start, which meant I didn't have to re-explain the requirements every time I started a new task. That continuity made the whole workflow smoother.